CSP-S 2022 (Senior)

2022-10-29, 14:30–18:30 (4 hours)

Problem 1: Holiday Plan (假期计划)

Input file `holiday.in`, output file `holiday.out`. Time limit: 2 s. Memory limit: 512 MB.

Little Bear's map has n points. The point numbered 1 is its home. The points numbered $2, 3, \ldots, n$ are all scenic spots. Some pairs of points are joined by a direct, bidirectional bus line. If there are direct lines between x and z_1, between z_1 and z_2, ..., between z_{k-1} and z_k, and between z_k and y, then we say that the trip between x and y is reachable with k transfers. In particular, if there is a direct line between x and y, we say that it is reachable with 0 transfers.

The holiday is coming soon. Little Bear plans to start from home, visit 4 distinct scenic spots, and return home after completing 5 legs: home → spot A → spot B → spot C → spot D → home, where each leg uses at most k transfers. There is no restriction on the points passed through while transferring: they may be the home or scenic spots, and the same point may be passed through repeatedly. For example, on the leg spot A → spot B, the points passed through while transferring may be the home, or spot C, or points passed through while transferring on the leg spot D → home.

Suppose every scenic spot has a score. Please help Little Bear plan a trip so that the sum of the scores of the four distinct scenic spots it visits is as large as possible.

Input

Read the input from the file `holiday.in`.

The first line contains 3 positive integers n, m, k, which are the number of points on the map, the number of pairs of points joined by a direct bidirectional line, and the maximum number of transfers on each leg, respectively.

The second line contains $n - 1$ positive integers, the scores of the scenic spots numbered $2, 3, \ldots, n$.

Each of the next m lines contains two positive integers x, y, meaning that points x and y are directly joined by a road. It is guaranteed that $1 \leq x, y \leq n$, and that there are no multiple edges and no self-loops.

Output

Write the output to the file `holiday.out`.

Output one positive integer: the maximum possible sum of the scores of the 4 distinct scenic spots that Little Bear visits.

Sample 1 input

```
8 8 1
9 7 1 8 2 3 6
1 2
2 3
3 4
4 5
5 6
6 7
7 8
8 1
```

Sample 1 output

```
27
```

Explanation of sample 1

When the planned trip is $1 \to 2 \to 3 \to 5 \to 7 \to 1$, the sum of the scores of the 4 scenic spots is $9 + 7 + 8 + 3 = 27$, and it can be shown that this is the maximum.

The trip $1 \to 3 \to 5 \to 7 \to 8 \to 1$ has score sum 24, and the trip $1 \to 3 \to 2 \to 8 \to 7 \to 1$ has score sum 25. Both satisfy the requirements, but their score sums are not the largest.

The trip $1 \to 2 \to 3 \to 5 \to 8 \to 1$ has score sum 30, but the leg $5 \to 8$ needs at least 2 transfers, so it violates the requirement of at most $k = 1$ transfers.

The trip $1 \to 2 \to 3 \to 2 \to 3 \to 1$ has score sum 32, but it does not visit 4 distinct scenic spots, so it does not satisfy the requirements either.

Sample 2 input

```
7 9 0
1 1 1 2 3 4
1 2
2 3
3 4
1 5
1 6
1 7
5 4
6 4
7 4
```

Sample 2 output

```
7
```

Sample 3

See `holiday/holiday3.in` and `holiday/holiday3.ans` in the contestant's directory.

Constraints

For all data, it is guaranteed that $5 \leq n \leq 2500$, $1 \leq m \leq 10000$, $0 \leq k \leq 100$, and that the score of every scenic spot satisfies $1 \leq s_i \leq 10^{18}$. It is guaranteed that at least one trip satisfying the requirements exists.

Test cases	$n \leq$	$m \leq$	$k \leq$
$1 \sim 3$	10	20	0
$4 \sim 5$	10	20	5
$6 \sim 8$	20	50	100
$9 \sim 11$	300	1000	0
$12 \sim 14$	300	1000	100
$15 \sim 17$	2500	10000	0
$18 \sim 20$	2500	10000	100

Problem 2: Strategy Game (策略游戏)

Input file `game.in`, output file `game.out`. Time limit: 1 s. Memory limit: 512 MB.

Little L and Little Q are playing a strategy game.

There is an array A of length n and an array B of length m. Based on them, a matrix C of size $n \times m$ is defined, satisfying $C_{ij} = A_i \times B_j$. All indices start from 1.

The game lasts q rounds. In each round, 4 parameters l_1, r_1, l_2, r_2 are given in advance, satisfying $1 \leq l_1 \leq r_1 \leq n$ and $1 \leq l_2 \leq r_2 \leq m$.

In the game, Little L first chooses an index x between l_1 and r_1, and then Little Q chooses an index y between l_2 and r_2. The score of the two players in this round is defined to be C_{xy}.

Little L's goal is to make this score as large as possible, and Little Q's goal is to make this score as small as possible. Both players are sufficiently clever and always play optimally.

Question: under the optimal strategies of the two players, what is the score of each round?

Input

Read the input from the file `game.in`.

The first line contains 3 positive integers n, m, q, which are the length of array A, the length of array B, and the number of rounds, respectively.

The second line contains n integers A_i, the elements of array A.

The third line contains m integers B_i, the elements of array B.

Each of the next q lines contains 4 positive integers, the values l_1, r_1, l_2, r_2 of that round.

Output

Write the output to the file `game.out`.

Output q lines, each containing one integer: the score of Little L and Little Q in that round under optimal play.

Sample 1 input

```
3 2 2
0 1 -2
```

```
-3 4
1 3 1 2
2 3 2 2
```

Sample 1 output

```
0
4
```

Explanation of sample 1

In this test, the matrix C is as follows:

```
0 0
-3 4
6 -8
```

In the first round, whether Little L chooses $x = 2$ or $x = 3$, Little Q can choose some y that makes the final score negative. Therefore choosing $x = 1$ is optimal for Little L, because then the score is certainly 0.

In the second round, since Little L can choose $x = 2$, Little Q can only choose $y = 2$, and the score is 4.

Sample 2 input

```
6 4 5
3 -1 -2 1 2 0
1 2 -1 -3
1 6 1 4
1 5 1 4
1 4 1 2
2 6 3 4
2 5 2 3
```

Sample 2 output

```
0
-2
3
2
-1
```

Sample 3

See `game/game3.in` and `game/game3.ans` in the contestant's directory.

Sample 4

See `game/game4.in` and `game/game4.ans` in the contestant's directory.

Constraints

For all data, $1 \le n, m, q \le 10^5$ and $-10^9 \le A_i, B_i \le 10^9$. For every round, $1 \le l_1 \le r_1 \le n$ and $1 \le l_2 \le r_2 \le m$.

Test cases	$n, m, q \le$	Special properties
1	200	1, 2
2	200	1
3	200	2
$4 \sim 5$	200	none
6	1000	1, 2
$7 \sim 8$	1000	1
$9 \sim 10$	1000	2
$11 \sim 12$	1000	none
13	10^5	1, 2
$14 \sim 15$	10^5	1
$16 \sim 17$	10^5	2
$18 \sim 20$	10^5	none

Here, special property 1 is: it is guaranteed that $A_i, B_i > 0$.

Special property 2 is: it is guaranteed that in every round, either $l_1 = r_1$ or $l_2 = r_2$.

Problem 3: Star War (星战)

Input file `galaxy.in`, output file `galaxy.out`. Time limit: 2 s. Memory limit: 512 MB.

In this round of the interstellar war, our side has built n outposts in the universe, joined by m one-way wormholes. We classify all wormholes whose endpoint is outpost u as the wormholes of outpost u.

Amid the flames of war these wormholes can hardly last long; enemy strikes may arrive at any time. The effective strikes among them fall into two kinds:

1. The enemy destroys some wormhole. This makes the two outposts it joins no longer directly reachable through this wormhole, but such a strike cannot destroy the two outposts it joins.

2. The enemy destroys some outpost. Since the main technology of a wormhole is concentrated at its exit, this causes all not-yet-destroyed wormholes of that outpost to be destroyed together. The wormholes starting from this outpost, however, are not destroyed.

Note: destruction only makes a wormhole unusable; it does not eliminate its existence.

To fight the enemy and maintain the links between the troops and the outposts, our side has developed two kinds of special forces responsible for repairing wormholes:

- Type A special forces can repair one specific wormhole.

- Type B special forces can repair all damaged wormholes of some outpost.

Considering the nature of the enemy's strikes, our side has not stockpiled too many strategic supplies at the outposts. Therefore, as long as some wormhole of an outpost has been repaired and is in a usable state, that outpost is also usable.

Our side has mastered a demanding spatial property. Using this property, our warships can teleport along wormholes into the enemy camp and carry out precision strikes.

To seize the best moment to launch a counteroffensive, the headquarters must watch every change on the battlefield, in order to find a moment at which a counteroffensive can be carried out. The commander-in-chief holds that:

- If, starting from any one of our outposts, with a suitably chosen route, one can travel through wormholes infinitely many times (the same outpost or the same wormhole may be passed through many times), then this outpost can counterattack.
- To make the wormhole travel continuous and reduce as much as possible the mass-energy loss of the warships when switching wormholes at an outpost, an outpost can travel continuously if and only if exactly one wormhole starting from that outpost is usable.
- If all of our outposts can counterattack and all of them can travel continuously, then this moment is an excellent moment for a counteroffensive.

The commander-in-chief gives you the order: according to the information fed back from the battlefield in real time, quickly tell him whether a counteroffensive can be carried out at the current moment.

Input

Read the input from the file `galaxy.in`.

The first line of the input contains two positive integers n, m.

Each of the next m lines contains two numbers u, v, meaning a wormhole starting from outpost u and going to outpost v. It is guaranteed that $u \neq v$, and that there are no two identical wormholes. Initially all wormholes and outposts are intact.

The next line contains one positive integer q, the number of queries.

Each of the next q lines describes a query or an operation. First read a positive integer t, the type of the instruction:

- If $t = 1$, the next two integers u, v mean that the enemy has destroyed the wormhole from outpost u to outpost v. It is guaranteed that this wormhole exists and has not been destroyed.
- If $t = 2$, the next integer u means that the enemy has destroyed outpost u. If all wormholes of this outpost have already been destroyed, this attack has no effect.
- If $t = 3$, the next two integers u, v mean that our side has repaired the wormhole from outpost u to outpost v. It is guaranteed that this wormhole exists and is destroyed.
- If $t = 4$, the next integer u means that our side has repaired outpost u. If this outpost has no destroyed wormhole, this repair has no effect.

After each instruction is executed, you need to decide whether a counteroffensive can be carried out. If so, output `YES`, otherwise output `NO`.

Output

Write the output to the file `galaxy.out`.

Output q lines in total. For each instruction, output whether a counteroffensive can be carried out after this instruction is executed.

Sample 1 input

```
3 6
2 3
2 1
1 2
1 3
3 1
3 2
11
1 3 2
1 2 3
1 1 3
1 1 2
3 1 3
3 3 2
2 3
1 3 1
3 1 3
4 2
1 3 2
```

Sample 1 output

```
NO
NO
YES
NO
YES
NO
NO
NO
YES
NO
NO
```

Explanation of sample 1

The states of the wormholes can be seen in the figure "Figure 1: explanation of sample 1" of the original statement, in which the edges are the wormholes that exist and have not been destroyed. (The figure is not reproduced here.)

Sample 2

See `galaxy/galaxy2.in` and `galaxy/galaxy2.ans` in the contestant's directory.

Sample 3

See `galaxy/galaxy3.in` and `galaxy/galaxy3.ans` in the contestant's directory.

Sample 4

See `galaxy/galaxy4.in` and `galaxy/galaxy4.ans` in the contestant's directory.

Constraints

For all data it is guaranteed that $1 \le n \le 5 \times 10^5$, $1 \le m \le 5 \times 10^5$, and $1 \le q \le 5 \times 10^5$.

Test cases	n	m	q	Special restrictions
1, 2, 3	≤ 10	≤ 20	≤ 50	none
4, 5, 6, 7, 8	$\le 10^3$	$\le 10^4$	$\le 10^3$	none
9, 10	$\le 5 \times 10^5$	$\le 5 \times 10^5$	$\le 5 \times 10^5$	guaranteed no $t = 2$ and no $t = 4$
11, 12	$\le 5 \times 10^5$	$\le 5 \times 10^5$	$\le 5 \times 10^5$	guaranteed no $t = 4$
13, 14, 15, 16	$\le 10^5$	$\le 5 \times 10^5$	$\le 5 \times 10^5$	none
17, 18, 19, 20	$\le 5 \times 10^5$	$\le 5 \times 10^5$	$\le 5 \times 10^5$	none

Problem 4: Data Transmission (数据传输)

Input file `transmit.in`, output file `transmit.out`. Time limit: 3 s. Memory limit: 1024 MB.

Little C is designing the routing system of a computer network.

The network used for testing has n hosts in total, numbered $1 \sim n$ in order. These n hosts are joined by $n - 1$ network cables; the i-th cable joins hosts a_i and b_i. It is guaranteed that any two hosts are connected, directly or indirectly, through finitely many cables. Limited by the transmission power, host a can directly transmit information to host b if and only if the two hosts are connected, directly or indirectly, through at most k cables.

In a computer network, data transmission often needs several forwardings. Suppose Little C needs to transmit data from host a to host b ($a \neq b$). Then it chooses some hosts used for the transmission, $c_1 = a, c_2, \cdots, c_{m-1}, c_m = b$, and forwards according to the following rule: for all $1 \le i < m$, host c_i sends the information directly to c_{i+1}.

Every host needs some time to process information; the i-th host needs v_i units of time to process information. Data travels through the network very quickly, so the transmission time can be ignored. Hence the time spent by the transmission process above is $\sum_{i=1}^{m} v_{c_i}$.

Now there are q data transmission requests in total; the i-th request sends data from host s_i to host t_i. Little C wants to know, for each request, the minimum number of units of time needed to complete the transmission.

Input

Read the input from the file `transmit.in`.

The first line of the input contains three positive integers n, Q, k, which are the number of hosts in the network, the number of requests, and the transmission parameter, respectively. The data guarantee $1 \le n \le 2 \times 10^5$, $1 \le Q \le 2 \times 10^5$, and $1 \le k \le 3$.

The second line of the input contains n positive integers; the i-th of them is v_i, and it is guaranteed that $1 \le v_i \le 10^9$.

Each of the next $n - 1$ lines contains two positive integers a_i, b_i, meaning a cable joining hosts a_i and b_i. It is guaranteed that $1 \le a_i, b_i \le n$.

Each of the next Q lines contains two positive integers s_i, t_i, meaning a request to send data from host s_i to host t_i. It is guaranteed that $1 \le s_i, t_i \le n$ and $s_i \neq t_i$.

Output

Write the output to the file `transmit.out`.

Q lines, each containing one positive integer: the minimum number of units of time needed by the i-th request during transmission.

Sample 1 input

```
7 3 3
1 2 3 4 5 6 7
1 2
1 3
2 4
2 5
3 6
3 7
4 7
5 6
1 2
```

Sample 1 output

```
12
12
3
```

Explanation of sample 1

For the first request, since hosts 4 and 7 need at least 4 cables to be connected, the data cannot be transmitted directly between the two hosts, and at least one forwarding is needed. We let it be forwarded once at host 1. It is not hard to see that host 1 needs only two cables to be connected to each of hosts 4 and 7, and the data processing time of host 1 is only 1, the smallest among all hosts, so the minimum transmission time is $4 + 1 + 7 = 12$.

For the third request, since hosts 1 and 2 need only 1 cable to be connected, transmitting the data directly is optimal, and the minimum transmission time is $1 + 2 = 3$.

Sample 2

See `transmit/transmit2.in` and `transmit/transmit2.ans` in the contestant's directory.

This sample satisfies the restrictions of test case 2.

Sample 3

See `transmit/transmit3.in` and `transmit/transmit3.ans` in the contestant's directory.

This sample satisfies the restrictions of test case 3.

Sample 4

See `transmit/transmit4.in` and `transmit/transmit4.ans` in the contestant's directory.

This sample satisfies the restrictions of test case 20.

Constraints

For all test data, $1 \le n \le 2 \times 10^5$, $1 \le Q \le 2 \times 10^5$, $1 \le k \le 3$, $1 \le a_i, b_i \le n$, $1 \le s_i, t_i \le n$, and $s_i \ne t_i$.

Test cases	n	Q	k	Special property
1	≤ 10	≤ 10	$= 2$	yes
2	≤ 10	≤ 10	$= 3$	yes
3	≤ 200	≤ 200	$= 2$	yes
4, 5	≤ 200	≤ 200	$= 3$	yes
6, 7	≤ 2000	≤ 2000	$= 1$	no
8, 9	≤ 2000	≤ 2000	$= 2$	no
10, 11	≤ 2000	≤ 2000	$= 3$	no
12, 13	$\le 2 \times 10^5$	$\le 2 \times 10^5$	$= 1$	no
14	$\le 5 \times 10^4$	$\le 5 \times 10^4$	$= 2$	yes
15, 16	$\le 10^5$	$\le 10^5$	$= 2$	no
17, 18, 19	$\le 2 \times 10^5$	$\le 2 \times 10^5$	$= 2$	no
20	$\le 5 \times 10^4$	$\le 5 \times 10^4$	$= 3$	yes
21, 22	$\le 10^5$	$\le 10^5$	$= 3$	no
23, 24, 25	$\le 2 \times 10^5$	$\le 2 \times 10^5$	$= 3$	no

Special property: it is guaranteed that $a_i = i + 1$, and b_i is chosen uniformly at random from $1, 2, \ldots, i$.

Source

The four problems of CSP-S 2022 (CCF CSP Senior, second round, held 2022-10-29), as hosted on QOJ:

- Problem 1: `https://qoj.ac/problem/5025`
- Problem 2: `https://qoj.ac/problem/5026`
- Problem 3: `https://qoj.ac/problem/5027`
- Problem 4: `https://qoj.ac/problem/5028`

The statements were translated from the official Chinese problem PDF embedded in these pages.